Software Quality Report: Roman Numeral Converter

USJ - Software Quality Course | Task #1: Assessing the quality of an app generated with AI Date: March 2026

---

1. Introduction

As established in Unit 1 of the course, quality is not only about **building the product right** but also about **building the right product**. This report analyses both dimensions: the correctness of the implementation and the degree to which the product meets the implicit quality standards expected of a modern web application.

---

2. Solution Overview

The solution follows a standard **client-server architecture**:

Layer	Technology	Role
Backend	ASP.NET Core Web API (.NET 10)	Conversion logic + REST API
Frontend	Angular 21 (standalone)	User interface + HTTP client

Core flow:

```
User input → Angular form → HTTP GET → ASP.NET Core API → Conversion logic
→ JSON response → Angular renders result
```

API endpoints:

- `GET /api/roman/to-roman?number=42` → `{ "roman": "XLII" }`
- `GET /api/roman/to-integer?roman=XLII` → `{ "integer": 42 }`

---

3. Quality Assessment Framework

Following the **ISO 9126 model** referenced in the course slides, software quality can be evaluated across multiple characteristics. This section maps the solution against the most relevant ones.

3.1 Functional Suitability (Correctness)

> *"Does the software do what it is supposed to do?"*

Conversion algorithm — Integer to Roman: The backend uses a greedy algorithm with a lookup table of 13 value-symbol pairs, explicitly including all subtractive combinations (CM, CD, XC, XL, IX, IV). This is a well-known, algorithmically correct approach.

Conversion algorithm — Roman to Integer: The service uses a linear scan with lookahead (subtractive notation detection) combined with a **cross-validation step**: after computing the integer, it converts it back to Roman and compares with the input. This elegantly rejects non-canonical forms such as `IIII`, `VV`, or `IIX` without needing a complex regex.

Input validation:

- Valid range: 1–3999 (enforced on both backend and frontend)
- Invalid characters rejected by the API with a descriptive `400 Bad Request`
- Frontend applies `Validators.min`, `Validators.max`, and `Validators.pattern` to catch issues before any HTTP call is made

Assessment: The functional correctness of the conversion logic is high. Both conversions handle edge cases and canonical form validation.

3.2 Reliability

> *"How well does the software perform its functions under stated conditions?"*

The backend is **stateless** — `RomanConverterService` is registered as a `Singleton` with no mutable state. This means there are no race conditions, no shared state issues, and the service is inherently thread-safe.

Error handling is implemented at the controller level:

```
catch (ArgumentOutOfRangeException ex) { return BadRequest(new { error =
ex.Message }); }
catch (ArgumentException ex)           { return BadRequest(new { error =
ex.Message }); }
```

Both the frontend and backend handle error states gracefully: the user sees a descriptive error message rather than a crash or a blank screen.

Known defect encountered during development: Angular's change detection was not triggered automatically after the HTTP observable resolved, causing the UI to only update after an interaction (clicking elsewhere). This was a **bug** — specifically a failure in the observable integration with Angular's zone. It was fixed by injecting `ChangeDetectorRef` and calling `detectChanges()` explicitly.

This illustrates the concept of **defect injection** described in Unit 2: even experienced developers make mistakes, and in this case the defect was injected during the component integration phase.

Assessment: Reliability is acceptable for a development/local environment. No retry logic or timeout handling is implemented for HTTP calls, which would be a concern in production.

3.3 Usability

> *"How easy is it for users to learn and use the software?"*

From the **user point of view** (as discussed in the course slides brainstorming session), quality means: easy to use, user friendly, fast.

Strengths:

- Two clearly separated conversion sections on one page — no navigation needed
- Inline validation messages appear when the user touches an invalid field
- The Convert button is disabled until input is valid, preventing unnecessary API calls
- Results are displayed prominently with large text and distinct styling

Weaknesses:

- No loading indicator while the HTTP request is in flight
- No way to clear/reset the result after a conversion
- The page has no explanation of Roman numeral rules for non-expert users
- No keyboard shortcut (e.g., Enter to submit)

Assessment: Usability is functional but minimal. The interface fulfils the basic task but lacks the polish expected of a production-ready product.

3.4 Maintainability

> *"How easy is it to modify or extend the software?"*

From the **developer point of view** (course slides), quality means: code easy to understand, architecture easy to maintain, clean and organised, modular, testable.

Separation of concerns: The backend cleanly separates responsibilities:

- `RomanConverterService` — pure business logic, no HTTP concerns
- `RomanController` — HTTP layer only, delegates all logic to the service
- `Program.cs` — infrastructure configuration (CORS, DI)

This separation makes the code **testable**: `RomanConverterService` can be unit tested in complete isolation without any HTTP context.

Frontend structure:

- `roman.service.ts` — all HTTP communication isolated in one place
- `app.ts` — only UI state and event handling
- `app.config.ts` — application-level providers

Technical debt identified:

- The `ChangeDetectorRef` workaround is a code smell. A cleaner solution would use Angular's `async` pipe or ensure proper zone integration, which would eliminate the need for manual change detection.
- No environment configuration file: the API URL `http://localhost:5000` is hardcoded in `roman.service.ts`. In a real project this should be in `environment.ts`.

Assessment: The architecture is clean and well-structured for its size. The main maintainability concern is the hardcoded API URL and the manual change detection workaround.

3.5 Portability and Deployability

> *"How easily can the software be moved to a different environment?"*

Strengths:

- Both technologies (.NET and Angular) are cross-platform
- The `npm start` script using `concurrently` allows launching both services with a single command
- The `prestart` script clears conflicting ports automatically

Weaknesses:

- No Docker or containerisation is used.

Assessment: Portability is low. The solution is tightly coupled to the developer's specific environment setup.

---

4. Defect Summary

Following the terminology from the course (Unit 1.3 — Key Definitions):

#	Type	Description	Status
1	**Bug**	Angular change detection not triggered after HTTP subscribe, causing stale UI	Fixed (`ChangeDetectorRef.detectChanges()`)
2	**Technical Debt**	API URL hardcoded in `roman.service.ts`	Open
3	**Technical Debt**	Absolute dotnet/ng paths in `package.json` reduce portability	Open
4	**Missing Feature**	No loading state indicator during HTTP call	Open
5	**Missing Feature**	No unit tests for conversion logic	Open

The bug #1 is a good example of what the course describes as a **failure** (observable incorrect behaviour to the end user) caused by a **defect** (incorrect change detection integration) which originates from a developer **error** (misunderstanding of Angular's zone mechanism when using HTTP observables).

---

5. Software Quality Assurance Activities Applied

As described in Unit 1.4, Software Quality Assurance encompasses activities that help ensure and improve quality. The following activities were applied during development:

SQA Activity	How it was applied

SQA Activity	How it was applied
Requirements analysis	The requirement was deliberately vague ("create a website"). Implicit requirements (valid range 1–3999, canonical Roman numerals, error handling) were identified and addressed proactively.
Architecture design	Separation of concerns was enforced from the start: backend logic in a service class, HTTP in the controller, frontend HTTP in a dedicated service.
Code review (informal)	The conversion logic was cross-validated programmatically (ToRoman ∘ ToInteger = identity) to catch defects algorithmically rather than through manual testing.
Defect removal	Bug #1 (change detection) was identified through manual exploratory testing and fixed.
Input validation	Validation was applied at two levels (frontend form validators + backend argument validation) as a **defect contention** measure — even if the frontend is bypassed, the backend will still reject invalid input.